

A Concise Derivation of UMAP

Uniform Manifold Approximation and Projection

Zhenwei Tang

Goal

Given high-dimensional data points

$$x_i \in \mathbb{R}^p, \quad i = 1, \dots, n,$$

UMAP aims to find a low-dimensional embedding

$$y_i \in \mathbb{R}^d, \quad d \ll p,$$

such that the *topological structure* of the data—captured by a weighted graph in the original space—is faithfully preserved in the embedding space.

The derivation proceeds in four stages:

1. Construct a fuzzy topological representation of the high-dimensional data.
2. Build an analogous fuzzy representation in the low-dimensional space.
3. Define a cost function that measures the discrepancy between the two representations.
4. Minimize the cost to obtain the embedding $\{y_i\}$.

Step 1: Local Metric Normalization

Why this step? Points in high-dimensional space are typically non-uniformly distributed. UMAP assumes the data lie on a Riemannian manifold that is *locally* approximately uniform. To enforce this assumption, the distance metric is normalized locally for each point.

For each point x_i , let ρ_i denote the distance to its nearest neighbor:

$$\rho_i = \min_{j \neq i} d(x_i, x_j),$$

where $d(\cdot, \cdot)$ is the ambient Euclidean distance. This ensures the nearest neighbor of every point has effective distance zero, anchoring the local scale.

Let k be the chosen number of neighbors. For each x_i , define the *local kernel weight* from x_i to x_j as

$$v_{j|i} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right),$$

where $\sigma_i > 0$ is a bandwidth parameter chosen so that the *effective number of neighbors* equals the perplexity target $\log_2 k$:

$$\sum_{j \neq i} v_{j|i} = \log_2 k.$$

Remark on σ_i . This condition is solved numerically (e.g., by binary search) for each i separately. The subtraction of ρ_i in the exponent ensures $v_{j^*|i} = 1$ for the nearest neighbor j^* , preventing the kernel from collapsing in sparse regions.

Step 2: Symmetrize into a Fuzzy Graph

Why this step? The weights $v_{j|i}$ are directed (asymmetric). To obtain an undirected graph suitable for a symmetric cost function, UMAP combines them using the *probabilistic union*:

$$w_{ij} = v_{j|i} + v_{i|j} - v_{j|i} \cdot v_{i|j}.$$

This formula is the fuzzy-set union: if A and B are two fuzzy membership values, then $A \cup B = A + B - A \cdot B$ (see Appendix ??).

The result is a symmetric weighted graph $G = (V, E, w)$ with $w_{ij} \in (0, 1]$. High w_{ij} means x_i and x_j are likely neighbors on the manifold.

Step 3: Low-Dimensional Similarity

Why this step? We need an analogous similarity measure in the low-dimensional embedding space $\mathbb{R}^d$.

For the embedding coordinates $y_i, y_j \in \mathbb{R}^d$, UMAP defines the low-dimensional similarity as a *Student-t* kernel (with one degree of freedom, matching t-SNE):

$$\hat{w}_{ij} = \left(1 + a \cdot \|y_i - y_j\|^{2b}\right)^{-1},$$

where $a > 0$ and $b > 0$ are hyperparameters fit to approximate a smooth step function. Specifically, they are chosen so that

$$\hat{w}_{ij} \approx \begin{cases} 1 & \text{if } \|y_i - y_j\| \leq \text{min_dist}, \\ \exp(-(\|y_i - y_j\| - \text{min_dist})) & \text{otherwise,} \end{cases}$$

where `min_dist` is a user-specified minimum distance in the embedding. This choice keeps nearby points tightly clustered while allowing global structure to spread out.

Default values. For the common setting `min_dist` = 0.1, the fit yields $a \approx 1.929$ and $b \approx 0.791$.

Step 4: The Cross-Entropy Cost Function

Why this step? UMAP treats w_{ij} and $\hat{w}_{ij}$ as Bernoulli probabilities of an edge existing between i and j in the high- and low-dimensional graphs, respectively. The natural measure of discrepancy between two Bernoulli distributions is the *binary cross-entropy* (see Appendix ??).

The total cost is summed over all pairs:

$$\mathcal{L}(\{y_i\}) = \sum_i \sum_{j \neq i} \left[w_{ij} \log \frac{w_{ij}}{\hat{w}_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - \hat{w}_{ij}} \right].$$

Dropping terms that do not depend on $\{y_i\}$ (i.e., terms involving $w_{ij} \log w_{ij}$ and $(1 - w_{ij}) \log(1 - w_{ij})$), the *effective* minimization objective reduces to:

$$\mathcal{L}(\{y_i\}) = - \sum_{(i,j) \in E} w_{ij} \log \hat{w}_{ij} - \sum_{(i,j) \notin E} (1 - w_{ij}) \log(1 - \hat{w}_{ij}).$$

Interpretation.

- The **first term** (attractive): for pairs that are close in the original space ($w_{ij} \approx 1$), penalize large $\|y_i - y_j\|$.
- The **second term** (repulsive): for pairs that are distant in the original space ($w_{ij} \approx 0$), penalize small $\|y_i - y_j\|$.

Step 5: Optimization via Stochastic Gradient Descent

Why this step? The full double sum over all $O(n^2)$ pairs is computationally prohibitive. UMAP uses *Stochastic Gradient Descent (SGD)* with *negative sampling* to approximate the gradients efficiently.

Gradient with respect to y_i . We compute $\nabla_{y_i} \mathcal{L}$.

Denote $r_{ij} = \|y_i - y_j\|^2$ and $\hat{w}_{ij} = (1 + a \cdot r_{ij}^b)^{-1}$.

Attractive gradient (for edge $(i, j) \in E$, contribution from the first term):

Applying the chain rule:

$$\frac{\partial}{\partial y_i} [-w_{ij} \log \hat{w}_{ij}] = -w_{ij} \cdot \frac{1}{\hat{w}_{ij}} \cdot \frac{\partial \hat{w}_{ij}}{\partial r_{ij}} \cdot \frac{\partial r_{ij}}{\partial y_i}.$$

Computing each factor:

$$\frac{\partial \hat{w}_{ij}}{\partial r_{ij}} = -ab r_{ij}^{b-1} \hat{w}_{ij}^2, \quad \frac{\partial r_{ij}}{\partial y_i} = 2(y_i - y_j).$$

Substituting:

$$\nabla_{y_i}^{\text{attr}} = \frac{2ab w_{ij} r_{ij}^{b-1}}{1 + a r_{ij}^b} \cdot (y_i - y_j).$$

Repulsive gradient (for non-edge $(i, j) \notin E$, contribution from the second term):

$$\frac{\partial}{\partial y_i} [-(1 - w_{ij}) \log(1 - \hat{w}_{ij})] = (1 - w_{ij}) \cdot \frac{1}{1 - \hat{w}_{ij}} \cdot \frac{\partial \hat{w}_{ij}}{\partial r_{ij}} \cdot \frac{\partial r_{ij}}{\partial y_i}.$$

Since for non-edges $w_{ij} \approx 0$:

$$\nabla_{y_i}^{\text{rep}} \approx \frac{-2b \hat{w}_{ij}}{(1 - \hat{w}_{ij})(1 + a r_{ij}^b)} \cdot r_{ij}^{b-1} \cdot (y_i - y_j).$$

Update rule. At each SGD step, for a sampled positive edge (i, j) and m sampled negative pairs $(i, k_1), \dots, (i, k_m)$:

$$y_i \leftarrow y_i - \eta \left(\nabla_{y_i}^{\text{attr}} + \sum_{\ell=1}^m \nabla_{y_i}^{\text{rep}, \ell} \right),$$

where η is a decaying learning rate. This reduces per-step cost from $O(n^2)$ to $O(m)$.

Conclusion

UMAP starts by modeling the high-dimensional data as a fuzzy topological graph with edge weights

$$w_{ij} = v_{j|i} + v_{i|j} - v_{j|i} \cdot v_{i|j},$$

derived from locally normalized distances. It then seeks low-dimensional coordinates $\{y_i\}$ such that the embedding's similarity structure

$$\hat{w}_{ij} = \left(1 + a\|y_i - y_j\|^{2b}\right)^{-1}$$

matches the original graph as closely as possible. The match is quantified by the cross-entropy cost

$$\mathcal{L} = - \sum_{(i,j) \in E} w_{ij} \log \hat{w}_{ij} - \sum_{(i,j) \notin E} (1 - w_{ij}) \log(1 - \hat{w}_{ij}),$$

which is minimized via SGD with negative sampling.

UMAP embeds data by minimizing the cross-entropy between a fuzzy high-dimensional graph and an analogous low-dimensional similarity structure, preserving local topology via locally normalized kernels.

Appendices

Appendix A: Fuzzy Set Union

In fuzzy set theory, membership values are real numbers in $[0, 1]$. For two fuzzy memberships $A, B \in [0, 1]$, the *fuzzy union* (probabilistic OR) is defined as:

$$A \cup B = A + B - A \cdot B.$$

This is the inclusion-exclusion formula for two independent Bernoulli events with probabilities A and B :

$$P(A \text{ or } B) = P(A) + P(B) - P(A)P(B).$$

Setting $A = v_{j|i}$ and $B = v_{i|j}$ gives the symmetrized edge weight w_{ij} .

Appendix B: Binary Cross-Entropy

For two Bernoulli distributions with parameters p and q , the *cross-entropy* of q relative to p is:

$$H(p, q) = -p \log q - (1 - p) \log(1 - q).$$

It measures how well the distribution q approximates p : $H(p, q) \geq H(p, p)$, with equality iff $p = q$ (Gibbs' inequality). In UMAP, $p = w_{ij}$ (high-dimensional) and $q = \hat{w}_{ij}$ (low-dimensional). Summing over all pairs gives the total cost $\mathcal{L}$.

Appendix C: Riemannian Manifold Assumption

UMAP assumes the data $\{x_i\}$ lie on (or near) a smooth Riemannian manifold $\mathcal{M} \subset \mathbb{R}^p$. A Riemannian manifold is a smooth d -dimensional manifold equipped with a smoothly varying inner product (metric tensor) g_x on each tangent space $T_x\mathcal{M}$.

Crucially, UMAP assumes the manifold is *locally uniform*: with respect to the Riemannian metric, data points are uniformly distributed. Since the true metric is unknown, UMAP approximates it by normalizing distances locally using ρ_i and σ_i , effectively learning a point-adaptive metric.

Appendix D: Stochastic Gradient Descent (SGD)

Standard gradient descent minimizes a loss $\mathcal{L}(\theta) = \sum_i \ell_i(\theta)$ by iterating:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta).$$

When the sum over all pairs is too expensive, *Stochastic Gradient Descent* approximates the full gradient by sampling a mini-batch $\mathcal{B}$ at each step:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \sum_{(i,j) \in \mathcal{B}} \ell_{ij}(\theta).$$

In UMAP, each positive edge is sampled once per epoch, and m negative pairs are sampled uniformly at random, giving an unbiased stochastic estimate of $\nabla \mathcal{L}$ per step. The learning rate η is typically annealed as $\eta(t) = \eta_0(1 - t/T)$ over T total epochs.

Appendix E: Negative Sampling

Computing the repulsive term exactly requires summing over all $O(n^2)$ non-edges. Negative sampling approximates this by, for each positive edge (i, j) , drawing m random non-neighbors $k_1, \dots, k_m$ uniformly and computing their repulsive gradients only. With m typically set to 5, this reduces cost from $O(n^2)$ to $O(nm)$ per epoch. The approach originates from Word2Vec and is valid because non-edges are the vast majority of pairs, so uniform sampling provides a low-variance gradient estimate.